

MisGastos

Aplicación web progresiva (PWA) para control de gastos personales

Documentación técnica y funcional
Junio 2026

1. Resumen ejecutivo

MisGastos es una aplicación de control de finanzas personales construida como una **Progressive Web App (PWA) de archivo único**, sin backend propio ni proceso de build. Está pensada para uso individual: registrar y visualizar gastos de tarjetas de crédito y cuentas de débito, controlar ciclos de facturación, categorizar automáticamente los movimientos, dividir gastos con otras personas (splits) y recibir recordatorios de pago y cobro.

La aplicación corre completamente en el navegador del usuario y persiste sus datos en `localStorage` del dispositivo. Se complementa con un proyecto externo de **Google Apps Script** que lee correos del banco BCI y entrega transacciones pendientes para sincronización automática.

¿A quién está dirigido?

- **Uso personal / unifamiliar:** fue diseñada y es usada por una sola persona (el dueño del proyecto), instalada como PWA en su iPhone.
- **Perfil del usuario objetivo:** alguien que maneja tarjetas de crédito con cuotas y ciclos de facturación en Chile (bancos BCI, Banco Chile, Scotiabank/Cencosud), que comparte gastos con otras personas (familia/pareja) y necesita llevar el registro de "quién le debe a quién".
- **No está pensada (en su forma actual) para multiusuario ni multi-dispositivo:** no tiene autenticación ni sincronización entre dispositivos; cada instalación tiene sus propios datos locales.

2. Stack técnico

Capa	Tecnología
Frontend	HTML5 + CSS3 + JavaScript vanilla (ES6+), sin frameworks (sin React/Vue/Angular)
Build	Ninguno — archivo único <code>index.html</code> servido tal cual
Librerías externas	<code>SheetJS / XLSX v0.18.5</code> (CDN cdnjs) para leer archivos Excel/CSV de cartolas bancarias
Persistencia	<code>localStorage</code> del navegador (sin base de datos)
PWA	Service worker propio (<code>sw.js</code>), meta tags <code>apple-mobile-web-app-*</code> para instalación en iOS
Hosting	Sitio estático (GitHub Pages) — repositorio <code>jbahamondez/misgastos</code>
Integración externa	Google Apps Script (proyecto separado, fuera del repo) + Google Sheets como cola de mensajes

3. Arquitectura general

El proyecto está compuesto por solo dos archivos relevantes en el repositorio:

- `index.html` (~2.430 líneas / ~137 KB) — contiene **todo**: estilos, marcado HTML de las 6 vistas y los 7 modales, y la lógica JavaScript completa.

- `sw.js` (~32 líneas) — service worker con estrategia **network-first** (intenta red, si falla usa caché) para que la app cargue rápido y funcione offline en iOS.

Estructura interna de `index.html`

1. **<head>**: metadatos PWA/iOS, viewport, y bloque `<style>` único con ~560 líneas de CSS (sistema de variables: colores, tipografía, espaciados, modo oscuro fijo).
2. **<body>** — Markup de las pantallas:
 - Dashboard (inicio)
 - Tarjetas (detalle por tarjeta de crédito)
 - Débito (cuenta corriente)
 - Historial (listado filtrable global)
 - Deudas (splits / por cobrar)
 - Ajustes (tarjetas, personas, categorías, reglas, respaldo)
 - Bottom navigation bar fija + botón flotante (FAB) para nuevo gasto
 - 7 modales: nuevo gasto, edición de transacción, división de compra (split), análisis de gastos, agregar/editar categoría, agregar regla de auto-categorización, importación de cartola
3. **<script>** final: toda la lógica de negocio en JavaScript plano, organizada en funciones top-level por área funcional (render de cada vista, CRUD de datos, cálculos de ciclos, sincronización, importación, etc.)

Modelo de despliegue

No hay pipeline de CI/CD ni archivos `.github/workflows`. El despliegue consiste en hacer push de `index.html` / `sw.js` al repositorio, y GitHub Pages sirve los archivos estáticos directamente. El usuario accede desde Safari en iPhone y la tiene instalada en pantalla de inicio como app standalone (`apple-mobile-web-app-capable`).

Nota sobre el service worker: usa estrategia network-first con fallback a caché (`misgastos-cache-v1`), y limpia versiones de caché antiguas en `activate`. Esto mitiga el cacheo agresivo de iOS que retrasaba la propagación de actualizaciones.

4. Modelo de datos (persistencia en localStorage)

Clave localStorage	Constante	Contenido
gastos_credito_v2	LS_C	Array de transacciones de tarjetas de crédito
gastos_debito_v2	LS_D	Array de transacciones de cuentas de débito
sueldo_mensual	LS_S	Sueldo/ingreso mensual del usuario
gastos_deudas_v1	LS_DEUDAS	Array de deudas/splits asociados a transacciones
deudas_personas_v1	LS_PERSONAS	Array de nombres de personas para dividir gastos
misgastos_categorias_v1	LS_CATS	Array de categorías (10 por defecto + personalizadas)
misgastos_cat_rules_v1	LS_CAT_RULES	Reglas de auto-categorización por palabra clave
misgastos_billing_dates	—	Día de cierre de facturación por tarjeta
misgastos_payment_dates	—	Fechas de pago registradas por tarjeta
misgastos_paid_flags	—	Flags de "ciclo pagado" por tarjeta/mes
misgastos_sync_last	—	Timestamp del último sync con Google Sheets (debounce)

Estructura de una transacción de crédito

```
{
  id: string,           // timestamp como id único
  cardId: 'bci' | 'bchile' | 'scotia',
  amount: number,       // monto total (puede ser negativo en anulaciones)
  desc: string,
  cuotas: number,       // 1, 3, 6, 12, 18, 24, 36, 48
  currency: 'CLP' | 'USD',
  date: string,         // ISO 8601, incluye hora
  source: 'manual' | 'cartola' | 'email_auto' | 'shortcut',
  catId: string,        // id de categoría (autocategorizada o manual)
  split?: [...],       // referencias a deudas si fue dividida
}
```

Estructura de una transacción de débito

```
{
  id: string,
  bank: 'bci' | 'bchile' | 'scotia',
  amount: number,       // negativo = anulación / reversa
  desc: string,
  currency: 'CLP',
}
```

```
date: string,          // ISO 8601 con hora
source: string,
catId: string
}
```

Estructura de una deuda/split

```
{
  id: string,           // id único de la deuda
  txId: string,         // referencia a la transacción original
  person: string,
  type: 'credito' | 'debito',
  amount: number,       // monto total de la transacción original
  deudaTotal: number,   // monto que debe esa persona
  deudaPerCuota: number, // monto por cuota (si aplica)
  desc: string,
  date: string,
  cuotas: number,
  cardId: string,
  paid: boolean,
  paidDate?: string
}
```

Tarjetas de crédito configuradas (hardcodeadas)

id	Banco	Cupo CLP	Cupo USD
bci	BCI	\$9.519.000	USD 470,74
bchile	Banco de Chile	\$9.519.000	USD 470,74
scotia	Scotiabank Cencosud	\$1.470.000	—

Categorías por defecto

 Super,  Comida,  Bencina,  Entretención,  Salud,  Hogar,  Ropa,  Tech,  Viajes,  Otros
— cada una con color asignado, totalmente editables/eliminables desde Ajustes.

5. Funcionalidades principales

5.1 Dashboard (Inicio)

- Widget central: monto a facturar próximo, días para el cierre, sueldo mensual (ocultable).
- 3 chips de resumen: total crédito, total débito, deudas "por cobrar" del ciclo anterior.
- Indicador de "Salud financiera": gasto total, ahorro estimado, distribución crédito/débito vs. sueldo.
- Sección expandible del ciclo anterior.
- Listado de últimas transacciones recientes.

5.2 Tarjetas de crédito

- Vista por tarjeta (BCI, Banco de Chile, Scotiabank): cupo comprometido, cargo del próximo cierre, disponible en CLP/USD.
- Barra de progreso de uso del cupo.
- Configuración de día de cierre por tarjeta (por defecto día 19).
- Cálculo de ciclos de facturación (ciclo actual y anterior) y prorrateo de cuotas.

5.3 Débito (cuenta corriente)

- Listado de movimientos de débito por banco.
- Estadísticas: total del período, promedio, mayor gasto, cantidad de movimientos.
- Soporte de montos negativos para anulaciones/reversas (se muestran en verde, descuentan del total).

5.4 Registro y edición de gastos

- Modal de "Nuevo gasto" con tabs Crédito / Débito.
- Campos: tarjeta o banco, monto, descripción, fecha/hora, categoría, moneda (CLP/USD), número de cuotas (crédito).
- Edición de cualquier transacción existente (modal de edición), incluida la posibilidad de modificar o eliminar su split.
- Eliminación de transacciones con confirmación y menú contextual (mantener presionado).

5.5 Categorías y auto-categorización

- CRUD completo de categorías (nombre, emoji, color) desde Ajustes.
- Reglas de auto-categorización por palabra clave: si la descripción contiene cierto texto, se asigna automáticamente una categoría.
- La auto-categorización se aplica a transacciones manuales, importadas por cartola y sincronizadas automáticamente vía Google Sheets.

5.6 Deudas y división de gastos (Split)

- Al registrar o importar un gasto, se puede dividir entre una o varias personas (splits parciales o por cuota).
- Gestión de personas (alta/baja) desde Ajustes.
- Vista "Deudas": agrupadas por persona y por ciclo (actual, anterior, antiguas), con filtros pendientes/pagadas/todas.
- Marcar deuda(s) como cobradas individualmente o por ciclo completo.
- Generar un resumen de texto de deudas para compartir (WhatsApp, etc.).
- Modificar o revertir un split ya aplicado a una transacción.

5.7 Recordatorios

- **Recordatorio de pago:** se muestra un modal cuando se acerca la fecha de facturación, indicando el monto a pagar; permite marcar la tarjeta como pagada.
- **Recordatorio de cobro:** en los primeros 5 días del mes, muestra un modal con las deudas pendientes del ciclo anterior agrupadas por persona, con botón para marcar "Cobrado".

5.8 Importación de cartola bancaria

- Soporta archivos `.xls`, `.xlsx` y `.csv` de BCI, Banco de Chile y Scotiabank.
- Detección automática de encabezados y parseo robusto de montos/fechas según formato de cada banco.
- Limpieza automática de descripciones (elimina prefijos típicos de BCI como "SAN CC", "DP **", etc.).
- Vista previa antes de confirmar la importación.
- Tras importar, ofrece dividir (split) cada transacción importada.

5.9 Sincronización automática vía Google Sheets / Apps Script

- Un proyecto externo de Google Apps Script (no incluido en el repo) escanea correos de notificación de compra del banco BCI (crédito y débito), extrae monto, comercio, fecha y hora, y los deja en una hoja de cálculo como "pendientes".
- La PWA consulta periódicamente (`action=getPending` , debounce de 15s) y crea automáticamente las transacciones correspondientes, aplicando auto-categorización.
- Tras procesarlas, marca las filas como "hechas" (`action=markDone`) para no duplicarlas.
- Soporta "anulaciones" (reversas de compra), registrándolas como montos negativos.
- Se dispara al abrir la app y al volver a foreground (`visibilitychange`).

Limitación conocida: Banco de Chile y Scotiabank no envían correos de notificación de compra, por lo que esta sincronización automática solo aplica a BCI. Para los otros bancos se recomienda importación de cartola periódica o registro manual/rápido.

5.10 Registro rápido vía URL (iOS Shortcuts)

- La app acepta parámetros de URL (`amount` , `desc` , `type` , `bank` , `cuotas` , `currency` , `date`) para crear una transacción instantáneamente sin abrir modales.

- Pensado para integrarse con la app "Atajos" (Shortcuts) de iOS y registrar gastos en segundos desde el celular.

5.11 Respaldo y restauración (JSON)

- Exportación de todos los datos (`localStorage`) a un archivo JSON descargable.
- Importación de ese archivo para restaurar el estado completo (con confirmación, ya que sobrescribe los datos actuales).
- Opción de borrar todos los datos locales con confirmación.

5.12 Historial y análisis

- Listado global de transacciones con filtros por tipo (todos / crédito / débito / banco específico) y por período (ciclo actual, mes actual, mes anterior, todo).
- Búsqueda por texto en la descripción.
- Modal de "Análisis": desglose de gastos por categoría con porcentajes, según período y tipo seleccionados.

6. Integraciones externas

Integración	Detalle
Google Apps Script ("MisGastosCola")	Proyecto separado (gestionado fuera del repo en script.google.com). Lee correos de Gmail con notificaciones de compra/anulación de BCI (crédito y débito), extrae monto, comercio, fecha y hora mediante expresiones regulares, y los almacena en una hoja de Google Sheets como cola de pendientes. Expone un endpoint web (<code>doGet</code> / <code>doPost</code>) consumido por la PWA vía <code>action=getPending</code> y <code>action=markDone</code> .
SheetJS (XLSX) v0.18.5	Cargado por CDN, usado solo para parsear archivos Excel de cartolas bancarias en el navegador.

7. Estado actual y evolución

La aplicación está en producción (uso diario real del dueño del proyecto) y se itera de forma incremental. Mejoras recientes incluyen: rediseño del dashboard, gestión completa de personas y splits desde Ajustes y desde el modal de edición, soporte de montos negativos (anulaciones), service worker para evitar problemas de caché en iOS, categorización automática también para transacciones sincronizadas, y recordatorio de cobro de deudas.

Temas en evaluación para el futuro (no implementados aún):

- Persistencia en una base de datos real (ej. Firebase/Supabase) o respaldo automático en la nube, manteniendo `localStorage` como almacenamiento principal por velocidad y soporte offline.
- Captura de movimientos de débito de Banco de Chile/Scotiabank (que no envían correo) vía importación periódica de cartola o registro rápido por Atajos de iOS.
- Actualización del ícono de la PWA en pantallas de inicio ya instaladas (limitación de iOS: requiere reinstalar el acceso directo).

8. Resumen de alcance

Incluido	No incluido (fuera de alcance actual)
• Registro manual y semi-automático de gastos crédito/débito • Gestión de ciclos de facturación y cuotas • Categorización (manual y automática) • División de gastos entre personas y seguimiento de cobros • Importación de cartolas y sync automático (BCI) • Recordatorios de pago y cobro • Respaldo/restauración manual vía JSON • Funcionamiento offline como PWA instalable	• Multiusuario / autenticación • Sincronización multi-dispositivo • Base de datos remota • Sync automático para Banco de Chile / Scotiabank • Notificaciones push propias • Reportes/exportación a Excel o PDF desde la app